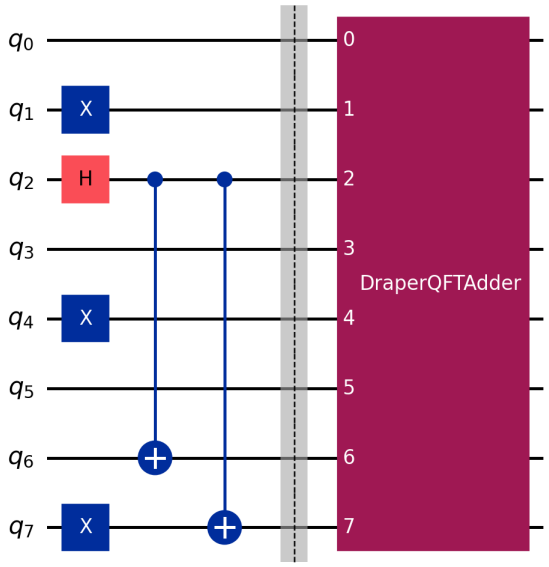
LAST LAB EXERCISE

Quantum teleportation comparison and QFT addition



Python implementation with Qiskit

Student	**Guillermo Aladro**
Subject	Introduction to Quantum Computing
Institution	ETSIST - Universidad Politecnica de Madrid
Academic year	2025-2026

Contents

1. Scope and limitations

This report implements the requested algorithms in Python with Qiskit. The ideal results use Qiskit Aer without noise. The comparison labelled noisy uses a local depolarizing and readout-error model. It is not an execution on an IBM Quantum processor because no IBM account credentials or job identifier were available.

The code is structured so that the local simulation is completely reproducible. A genuine hardware run should be added by submitting the same measured circuit through the student's IBM Quantum account and replacing the local-noise discussion with the retrieved job results.

2. Teleportation: ideal and noisy simulations

Each teleported state is verified by applying the inverse of its preparation to Bob's output. A perfect teleportation therefore produces measurement 0. The ideal simulator reaches success probability 1 for all states. Noise lowers the success rate because gate errors accumulate in entanglement generation, Bell measurement, conditional correction and final verification.

Input	Ideal success	Noisy success
|0>	1.000	0.916
|1>	1.000	0.926
|+>	1.000	0.925
|->	1.000	0.913
|i>	1.000	0.894
|-i>	1.000	0.894

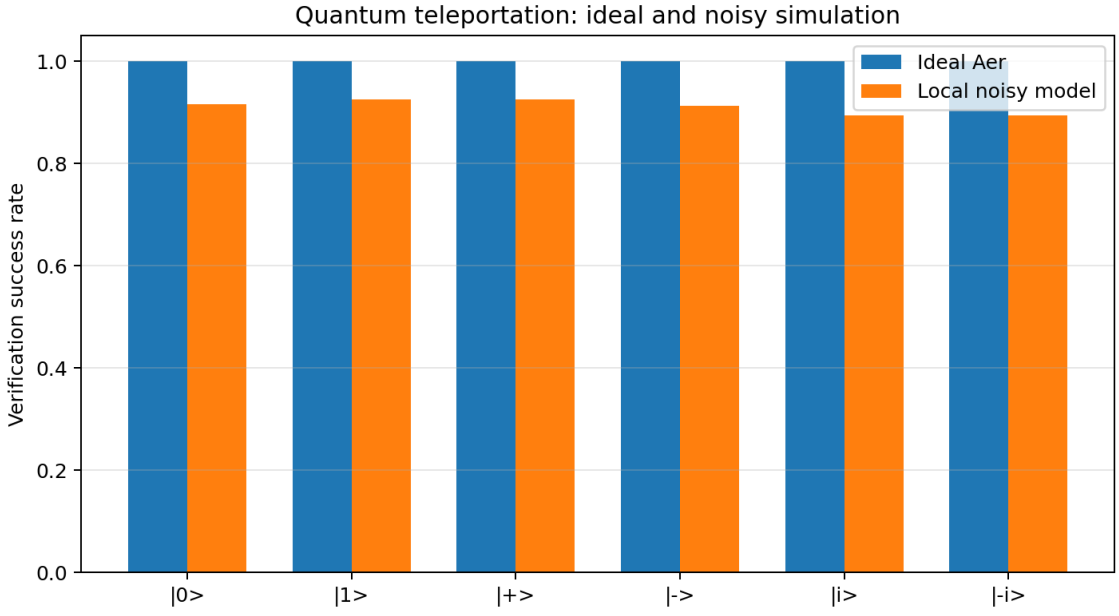


Figure 1. Teleportation verification on ideal and local noisy simulators.

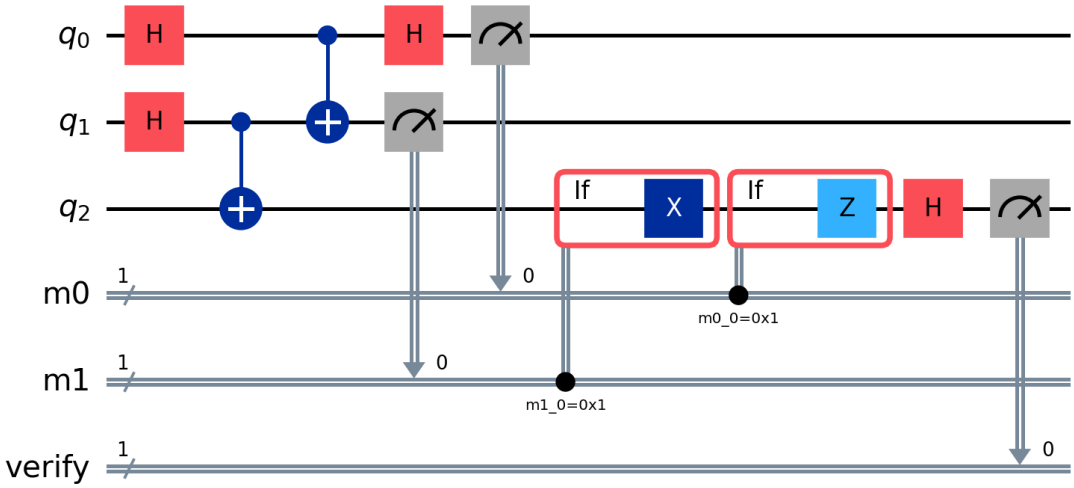


Figure 2. Dynamic verification circuit for the teleported |+> state.

3. Four-qubit QFT addition

Two four-qubit registers are used. Register a is preserved and added to register b modulo 16. The input is the entangled state (|2>|9> + |6>|5>)/sqrt(2). In both branches the arithmetic result is 11, so the expected output is (|2>|11> + |6>|11>)/sqrt(2).

The implementation appends Qiskit's DraperQFTAdder. Internally, the target register is transformed to the Fourier basis, controlled phase rotations add the value of a, and an inverse QFT returns the target to the computational basis.

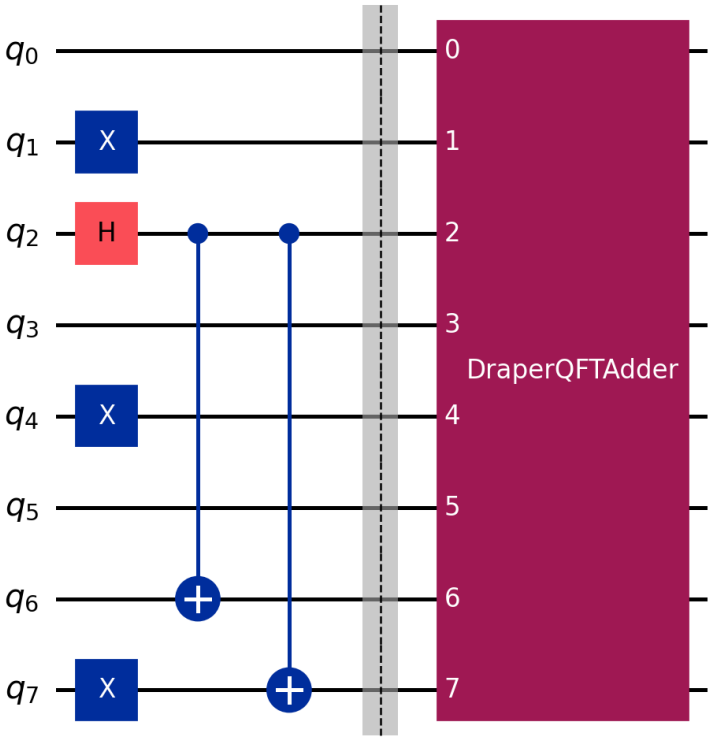


Figure 3. Eight-qubit circuit with entangled input and Draper QFT adder.

Metric	Result
Ideal state fidelity	1.000000000000

Metric	Result
Ideal P(b=11)	1.0000
Noisy P(b=11)	0.4150
Expected bit strings	10110010, 10110110

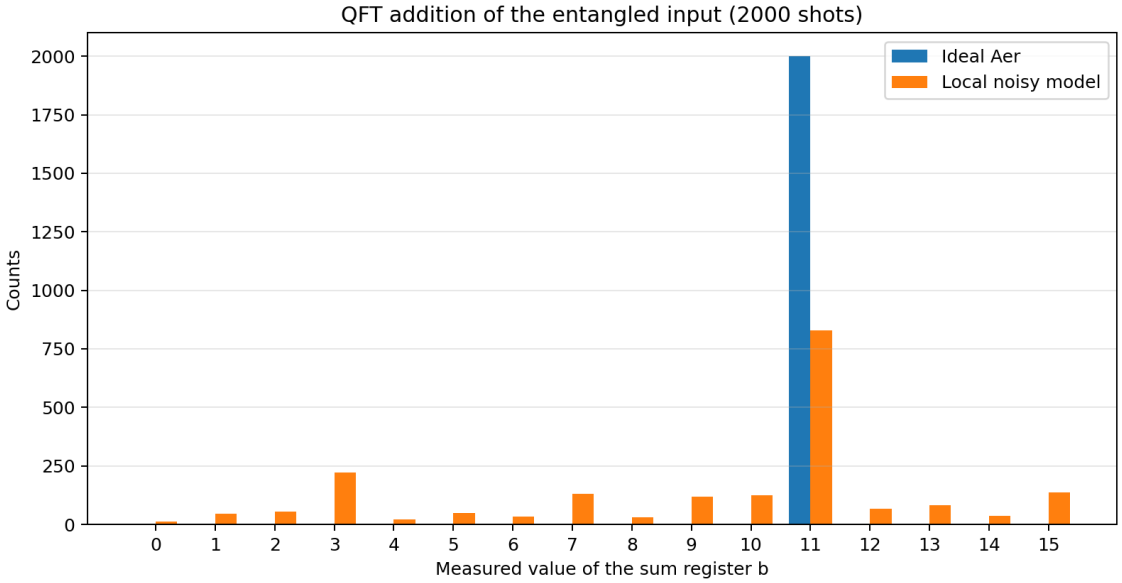


Figure 4. Marginal distribution of the four-qubit sum register.

3.1 Entangled input preparation

```
    # Shared 1-bits: a1 and b0. Branch 0 also has b3=1.
    qc.x(1)
    qc.x(4)
    qc.x(7)
    # q2 distinguishes a=2 from a=6 and controls the correlated b changes.
    qc.h(2)
    qc.cx(2, 6)
    qc.cx(2, 7)


def qft_adder_circuit(measured: bool = True) -> QuantumCircuit:
    qc = QuantumCircuit(8, 8 if measured else 0, name="4-qubit QFT adder")
    prepare_entangled_input(qc)
    qc.barrier()
    qc.append(DraperQFTAdder(4, kind="fixed"), range(8))
    if measured:
```

4. Discussion of ideal versus hardware-like results

The ideal simulator produces only the two expected eight-bit strings, and both have b=11. The state fidelity is one within numerical precision. In the local noisy simulation, probability leaks to neighboring and unrelated outputs. The QFT adder contains many controlled rotations after decomposition, so two-qubit errors and readout errors have a much larger cumulative effect than in a short circuit.

A real IBM processor would show device-specific behaviour: restricted connectivity, calibration-dependent gate errors, decoherence during the circuit, crosstalk and asymmetric readout. Consequently, the exact histogram cannot be predicted from this generic model. The local result should be interpreted only as a controlled demonstration of the qualitative difference between perfect and noisy computation.

5. Conclusions

The QFT adder correctly maps each branch of the entangled input to the same sum while preserving coherence in register a. The ideal result agrees exactly with theory. The noisy simulation demonstrates why circuit depth, two-qubit gates and readout mitigation are central when moving from an ideal simulator to physical hardware.

6. Files and execution

Run `python iqc7_qft_addition.py`. The script creates `IQC7_results.json` and `result_qft_addition.json`. The assignment and QFT lecture slides are included as references. Real IBM execution requires the student's credentials and cannot be fabricated offline.